

Snort

1) Introduction

This Lab demonstrates the deployment of **Snort** as both an **Intrusion Detection System (IDS)** and an **Intrusion Prevention System (IPS)** within a virtual laboratory environment.

The primary goal of this lab is to monitor network traffic, identify suspicious or malicious activity, and enforce preventive security controls through custom Snort rules.

By configuring Snort to operate in both IDS and IPS modes, this lab highlights the functional differences between detection-only monitoring and active traffic prevention, showing how each mode contributes to strengthening network security.

Benefits of Using Snort as IDS and IPS:

Snort is a widely used open-source network security tool that provides enhanced visibility and protection for network environments.

When deployed as an **Intrusion Detection System (IDS)**, Snort passively monitors network traffic and inspects packets against predefined rule sets. Suspicious activities are logged and reported through alerts, enabling security analysts to detect potential attacks and investigate incidents without disrupting network communication.

When deployed as an **Intrusion Prevention System (IPS)**, Snort operates inline with network traffic and actively enforces security policies. In this mode, malicious packets are dropped in real time before reaching internal systems, providing immediate protection against threats.

Overall, Snort improves network visibility, supports incident response efforts, and strengthens the overall security posture of the monitored environment.

2) Network Topology

Lab Environment

The lab environment was created using **VMware** and consists of two virtual machines connected to the same internal network:

- **Ubuntu Virtual Machine**
 - Role: Snort Sensor (IDS / IPS)
 - Network Interface: **ens33**
 - IP Address: 192.168.43.128

- **Windows 10**
 - Role: Client / Traffic Generator
 - IP Address: 192.168.43.129
 -
- **Protected Network (Host-Only / NAT in VMware):**
 - Ip Address: 192.168.43.0/24
 - Gateway: 192.168.43.1

In this setup, the Ubuntu virtual machine monitors and protects the internal network using Snort, while the Windows virtual machine generates test traffic such as ICMP pings and HTTP requests to validate detection and prevention functionality.

3) Snort Installation & Configuration

Install Snort:

```

43 packages can be upgraded. Run 'apt list --upgradable' to see them.
maram@maram:~$ sudo apt install snort
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libdaq2t64 libdumbnet1 liblua5.1-2 liblua5.1-common
  libnetfilter-queue1 libpcrc3 oinkmaster snort-common snort-common-libraries
  snort-rules-default
Suggested packages:
  snort-doc
The following NEW packages will be installed:
  libdaq2t64 libdumbnet1 liblua5.1-2 liblua5.1-common
  libnetfilter-queue1 libpcrc3 oinkmaster snort-common
  snort-common-libraries snort-rules-default
0 upgraded, 11 newly installed, 0 to remove and 43 not upgraded.
Need to get 2,666 kB of archives.
After this operation, 11.4 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://eg.archive.ubuntu.com/ubuntu noble/universe amd64 liblua5.1-com
mon all 2.1.0+git20231223.c525bcb+dfsg-1 [49.2 kB]
Get:2 http://eg.archive.ubuntu.com/ubuntu noble/universe amd64 liblua5.1-2 a
md64 2.1.0+git20231223.c525bcb+dfsg-1 [275 kB]
Get:3 http://eg.archive.ubuntu.com/ubuntu noble/universe amd64 libpcrc3 amd64 2:

```

Configure HOME_NET

In `/etc/snort/snort.conf`

```

GNU nano 7.2 /etc/snort/snort.conf *
# value of DEBIAN_SNORT_HOME_NET is defined in the
# /etc/snort/snort.debian.conf configuration file
#
ipvar HOME_NET 192.168.43.0/24

# Set up the external network addresses. Leave as "any" in most situations
ipvar EXTERNAL_NET any
# If HOME_NET is defined as something other than "any", alternative, you can
# use this definition if you do not want to detect attacks from your internal
# IP addresses:
#ipvar EXTERNAL_NET !$HOME_NET

# List of DNS servers on your network
ipvar DNS_SERVERS $HOME_NET

# List of SMTP servers on your network
ipvar SMTP_SERVERS $HOME_NET

# List of web servers on your network
ipvar HTTP_SERVERS $HOME_NET

^G Help      ^O Write Out ^M Where Is   ^K Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^L Replace  ^U Paste    ^J Justify  ^_ Go To Line

```

Local Rules:

```
maram@maram:~$ sudo nano /etc/snort/snort.conf
[sudo] password for maram:
maram@maram:~$
```

```
GNU nano 7.2 /etc/snort/rules/local.rules *
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# -----
# LOCAL RULES
# -----
# This file intentionally does not come with signatures.  Put your local
# additions here.
# IDS rule - Detect ICMP ping to HOME_NET
alert icmp any any -> $HOME_NET any (msg:"IDS: ICMP Ping detected"; sid:1000001; rev:1;)

# IDS rule - Detect HTTP access to HOME_NET on port 80
alert tcp any any -> $HOME_NET 80 (msg:"IDS: HTTP access detected"; sid:1000002; rev:1;)

# IPS rule - Block Telnet access to HOME_NET on port 23
drop tcp any any -> $HOME_NET 23 (msg:"IPS: Block Telnet access"; sid:1000003; rev:1;)

# IPS rule - Block ICMP ping to HOME_NET (Inline Mode)
alert icmp any any -> $HOME_NET any (msg:"IDS: ICMP Ping detected"; sid:1000004; rev:1;)
drop icmp any any -> $HOME_NET any (msg:"IPS: ICMP Ping blocked"; sid:1000005; rev:1;)

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   ^M-U Undo     ^M-A Set M
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^D Justify    ^/_ Go To Line  ^M-E Redo     ^M-G Copy
```

Explanation of Snort Rules

1. ICMP Ping Detection (IDS)
This rule detects ICMP echo request packets (ping) sent from any source to the protected network (HOME_NET). When a ping is detected, Snort generates an alert without blocking the traffic. This rule is used to monitor reconnaissance activity.
2. HTTP Traffic Detection (IDS)
This rule detects TCP traffic targeting port 80 (HTTP) on HOME_NET. It alerts whenever a system attempts to access a web service, providing visibility into HTTP communication without preventing it.
3. Telnet Blocking (IPS)
This rule blocks all TCP traffic directed to port 23 (Telnet) on HOME_NET. Telnet is an insecure protocol, and this rule prevents unauthorized or insecure remote access by dropping the packets in inline mode.
4. ICMP Ping Alert (Inline Monitoring)
This rule detects ICMP ping traffic while Snort is running in inline mode and generates an alert for logging and monitoring purposes. It provides visibility into ICMP activity even when IPS functionality is enabled.
5. ICMP Ping Blocking (IPS)
This rule actively blocks ICMP ping requests sent to HOME_NET by dropping the packets. It prevents attackers from using ping for network discovery and confirms Snort's ability to operate as an Intrusion Prevention System.

4) Snort as IDS Mode

Snort is first run in **IDS mode** to only detect and log attacks.

```
maram@maram: ~  
maram@maram:~$ sudo snort -c /etc/snort/snort.conf -i ens33 -A fast  
[sudo] password for maram:  
Running in IDS mode  
  
--== Initializing Snort ==--  
Initializing Output Plugins!  
Initializing Preprocessors!  
Initializing Plug-Ins!  
Parsing Rules file "/etc/snort/snort.conf"  
PortVar 'HTTP_PORTS' defined : [ 80:81 311 383 591 593 901 1220 1414 1741 1830  
2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777  
7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300  
8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002  
55555 ]  
PortVar 'SHELLCODE_PORTS' defined : [ 0:79 81:65535 ]  
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]  
PortVar 'SSH_PORTS' defined : [ 22 ]  
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]  
PortVar 'SIP_PORTS' defined : [ 5060:5061 5600 ]  
PortVar 'FILE_DATA_PORTS' defined : [ 80:81 110 143 311 383 591 593 901 1220 14  
14 1741 1830 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:71  
45 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 82  
43 8280 8300 8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444  
41080 50002 55555 ]
```

Testing from Windows VM

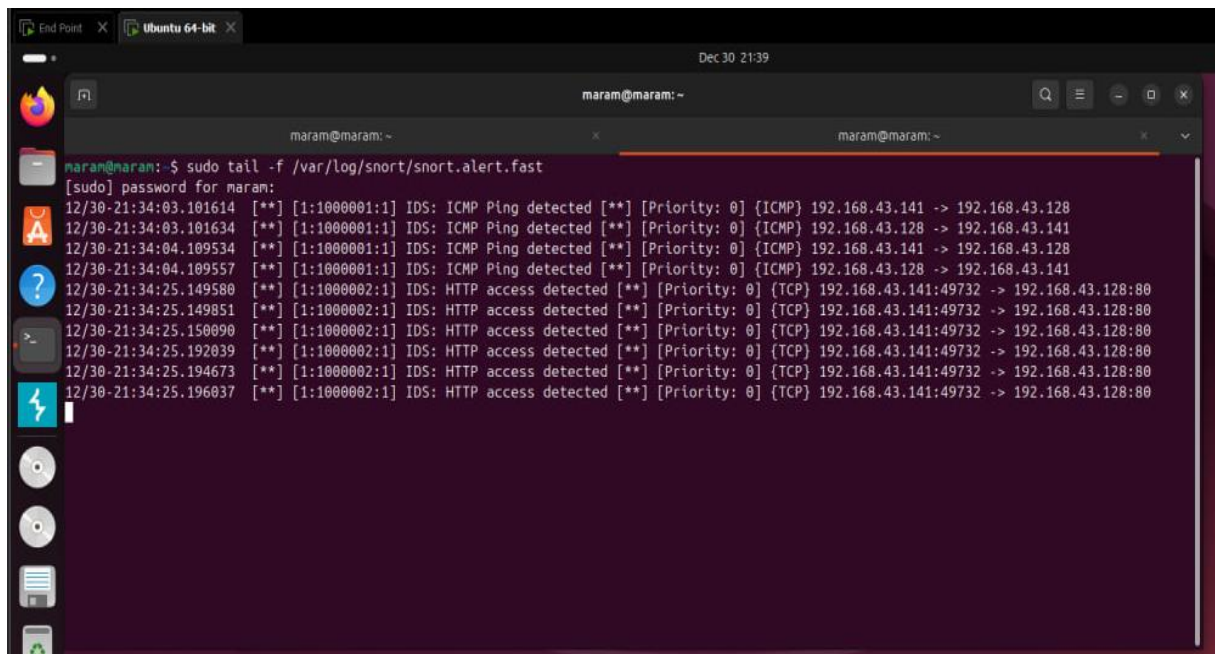
The screenshot shows a Windows command prompt window with the title "Administrator: Command Prompt". The user has entered several commands:

- `C:\Windows\system32>ping 192.168.43.128`: This command is highlighted with a yellow box. The output shows four successful ping replies from 192.168.43.128, each with 32 bytes of data, a time of less than 1ms, and a TTL of 64.
- `C:\Windows\system32>curl http://192.168.43.128`: This command is also highlighted with a yellow box. The output shows the HTML content of the page, which is a default Ubuntu page. The page title is "Apache2 Ubuntu Default Page: It works". The page content includes a modified date of 2022-03-22 and a link to a bug report.

The command prompt window is partially obscured by a smaller, semi-transparent window in the bottom right corner, which also has the title "Administrator: Command Prompt".

Result: Logs and Alerts Monitoring

Traffic is detected and logged only, but not blocked, because Snort is running in IDS mode.

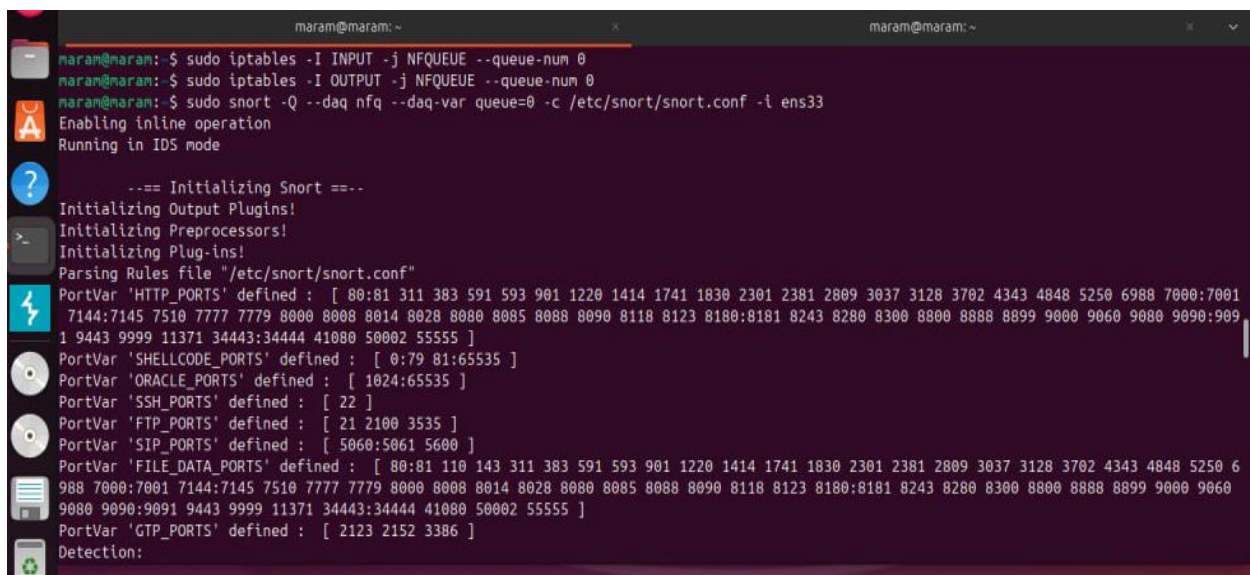


A terminal window on an Ubuntu 64-bit system showing the output of the command `sudo tail -f /var/log/snort/snort.alert.fast`. The terminal displays a series of Snort alerts. The first three alerts are ICMP Ping detections from 192.168.43.141 to 192.168.43.128. The subsequent five alerts are HTTP access detections from 192.168.43.141 to 192.168.43.128:80. All alerts are marked with a double asterisk (**) indicating a high confidence level.

```
maram@maram:~$ sudo tail -f /var/log/snort/snort.alert.fast
[sudo] password for maram:
12/30-21:34:03.101614  ** [1:1000001:1] IDS: ICMP Ping detected  ** [Priority: 0] {ICMP} 192.168.43.141 -> 192.168.43.128
12/30-21:34:03.101634  ** [1:1000001:1] IDS: ICMP Ping detected  ** [Priority: 0] {ICMP} 192.168.43.128 -> 192.168.43.141
12/30-21:34:04.109534  ** [1:1000001:1] IDS: ICMP Ping detected  ** [Priority: 0] {ICMP} 192.168.43.141 -> 192.168.43.128
12/30-21:34:04.109557  ** [1:1000001:1] IDS: ICMP Ping detected  ** [Priority: 0] {ICMP} 192.168.43.128 -> 192.168.43.141
12/30-21:34:25.149580  ** [1:1000002:1] IDS: HTTP access detected  ** [Priority: 0] {TCP} 192.168.43.141:49732 -> 192.168.43.128:80
12/30-21:34:25.149851  ** [1:1000002:1] IDS: HTTP access detected  ** [Priority: 0] {TCP} 192.168.43.141:49732 -> 192.168.43.128:80
12/30-21:34:25.150090  ** [1:1000002:1] IDS: HTTP access detected  ** [Priority: 0] {TCP} 192.168.43.141:49732 -> 192.168.43.128:80
12/30-21:34:25.192039  ** [1:1000002:1] IDS: HTTP access detected  ** [Priority: 0] {TCP} 192.168.43.141:49732 -> 192.168.43.128:80
12/30-21:34:25.194673  ** [1:1000002:1] IDS: HTTP access detected  ** [Priority: 0] {TCP} 192.168.43.141:49732 -> 192.168.43.128:80
12/30-21:34:25.196037  ** [1:1000002:1] IDS: HTTP access detected  ** [Priority: 0] {TCP} 192.168.43.141:49732 -> 192.168.43.128:80
```

5) Snort as IPS Mode (Inline Mode)

Configure iptables on Ubuntu & Run Snort in IPS Mode

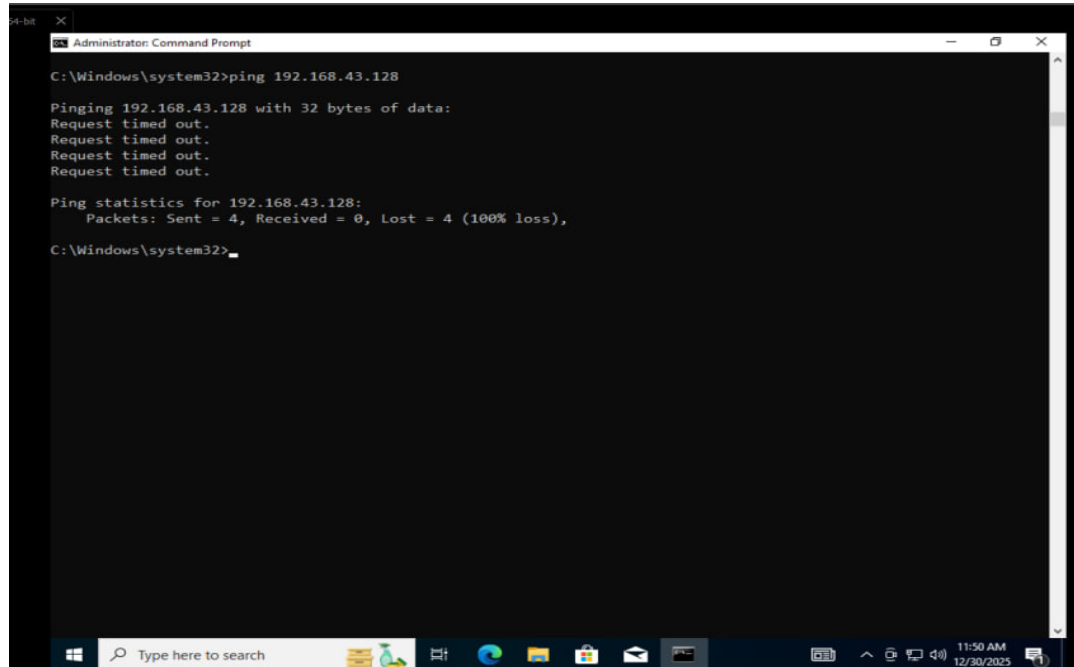


A terminal window showing the configuration of Snort for IPS mode. The user runs `sudo iptables -I INPUT -j NFQUEUE --queue-num 0` and `sudo iptables -I OUTPUT -j NFQUEUE --queue-num 0` to set up iptables. Then, `sudo snort -Q --daq nfq --daq-var queue=0 -c /etc/snort/snort.conf -l ens33` is executed. The terminal output shows the initialization of Snort, including parsing the rules file and defining various port sets like HTTP_PORTS, SHELLCODE_PORTS, ORACLE_PORTS, SSH_PORTS, FTP_PORTS, SIP_PORTS, FILE_DATA_PORTS, and GTP_PORTS. The output ends with the word "Detection:".

```
maram@maram:~$ sudo iptables -I INPUT -j NFQUEUE --queue-num 0
maram@maram:~$ sudo iptables -I OUTPUT -j NFQUEUE --queue-num 0
maram@maram:~$ sudo snort -Q --daq nfq --daq-var queue=0 -c /etc/snort/snort.conf -l ens33
Enabling inline operation
Running in IDS mode

--== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
PortVar 'HTTP_PORTS' defined : [ 80:81 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001
7144:7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000 9060 9080 9090:909
1 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined : [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060:5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80:81 110 143 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037 3128 3702 4343 4848 5250 6
988 7000:7001 7144:7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000 9060
9080 9090:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'GTP_PORTS' defined : [ 2123 2152 3386 ]
Detection:
```

Test Blocking from Windows VM

A screenshot of a Windows VM's command prompt window. The window title is "Administrator: Command Prompt". The command prompt shows the execution of the command "ping 192.168.43.128". The output indicates that the ping failed with "Request timed out." for four consecutive attempts. The ping statistics show "Packets: Sent = 4, Received = 0, Lost = 4 (100% loss)". The command prompt is running in the "C:\Windows\system32" directory. The Windows taskbar is visible at the bottom, showing the search bar, task view, and various application icons. The system clock in the bottom right corner shows "11:50 AM 12/30/2025".

```
C:\Windows\system32>ping 192.168.43.128

Pinging 192.168.43.128 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.43.128:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\Windows\system32>
```

Result: Logs and Alerts Monitoring

In inline IPS mode, Snort may drop packets without generating alerts by default. Alert logging was explicitly enabled to verify packet blocking behavior.

7) Use Case Scenarios

- Detect ICMP ping attempts used for network reconnaissance.
- Identify HTTP traffic accessing internal services.
- Block insecure services such as Telnet to prevent unauthorized access.
- Monitor suspicious inbound traffic targeting the protected network.
- Enforce basic network security policies using custom Snort rules.

8) Key Advantages

- Open-source and widely adopted in security environments.
- Supports both IDS and IPS functionality.
- Provides real-time traffic inspection and alerting.
- Allows custom rule creation for specific security needs.
- Integrates easily with SIEM and log analysis tools.
- Suitable for learning, labs, and enterprise environments.

9) Challenges & Limitations

- Requires proper rule tuning to reduce false positives.
- Inline IPS mode may introduce performance overhead.
- Manual rule management can be time-consuming.
- Incorrect HOME_NET configuration may cause missed detections.
- Needs correct network placement to be fully effective.

10) Conclusion

Snort proved to be an effective open-source solution for both intrusion detection and intrusion prevention. In IDS mode, it successfully monitored and logged ICMP and HTTP traffic, providing visibility into network activity.

In IPS inline mode, Snort actively blocked ICMP ping traffic, demonstrating its ability to prevent malicious behavior in real time.

This lab confirms that Snort is a flexible and powerful tool for enhancing network security in virtual environments.